

chapter3 报告

陈毓椿 计36 2023010452

功能实现

- `syscall_times`: 在 `TaskControlBlock(os/src/task/task.rs)` 结构中添加了 `syscall_times` 数组, 用于对不同 id 系统调用的计数。
 - `MAX_SYSCALL_NUM` 用于表示支持的系统调用 ID 范围, 参考目前实现的最大 ID 为 `sys_trace` 的 410, 实验中设置 512 恰好覆盖并预留部分空间
 - 在 `os/src/task/mod.rs` 完成初始化
- `record_syscall` & `get_syscall_times`: 在 `os/src/task/mod.rs` 实现上述两个功能函数, `record_syscall` 用于单次调用时计数, `get_syscall_times` 用于从给定 id 读取调用次数。在每次 `sys_call` 的开头, 会首先执行一次 `record_syscall` 记录。
- `sys_trace`: 根据实验文档实现 `sys_trace` 系统调用, 调用 `get_syscall_times` 完成 id 为 2 时的调用次数返回。

问答题

1. 运行下述脚本, 得到 `TEST=2` 时的 3 个 bad 测例结果如下:

```
make run TEST=2 BASE=1

# 使用的 SBI: RustSBI
# SBI 版本: ch1 提供的 rustsbi-qemu.bin

...
[kernel] PageFault in application, bad addr = 0x0, bad instruction =
0x804003a4, kernel killed it.
[kernel] IllegalInstruction in application, kernel killed it.
[kernel] IllegalInstruction in application, kernel killed it.
...
```

对应的测例行为及其解释如下:

- `ch2b_bad_address` - 触发非法内存访问异常, 符合输出 `[kernel] PageFault in application, bad addr = 0x0, bad instruction = 0x804003a4, kernel killed it.`
- `ch2b_bad_instructions` - 尝试执行S态特权指令 `sret`, 应该报错, 符合输出 `[kernel] IllegalInstruction in application, kernel killed it.`
- `ch2b_bad_register` - 尝试读取S态特权寄存器 `sstatus`, 应该报错, 符合输出 `[kernel] IllegalInstruction in application, kernel killed it.`

2. 深入理解 `trap.S` 中两个函数 `__alltraps` 和 `__restore` 的作用

1. L40:

刚进入 `__restore` 时, `sp` 指向的是内核栈上的 **TrapContext 首地址** (即内核栈分配 34×8 字节后的栈顶), 这里保存了寄存器、`sstatus`、`sepc`等。

`__restore` 的两种使用情景:

- **Trap 处理返回:** Trap 的处理流程中, 先后是用户程序触发 trap $\rightarrow$ `__alltraps` 保存上下文 $\rightarrow$ `trap_handler` 处理完毕返回 $\rightarrow$ fall through 到 `__restore`。此时 `sp` 就是 `__alltraps` 中 `addi sp, sp, -34*8` 后的值, 指向刚保存的 `TrapContext`, `__restore` 将负责恢复这些 `TrapContext`。
- **首次启动/任务切换时恢复:** 在 `os/src/task/context.rs` 中, `goto_restore` 将 `ra` 设为 `__restore` 地址, `sp` 设为 `init_app_cx` 返回的内核栈指针。这一步为首次启动与任务切换手动构造了一个好的 `TrapContext` 并指定好了返回地址, 当 `__switch` 执行 `ret` 时跳转到 `__restore`, `sp` 指向预先构造好的 `TrapContext`, 随后进行 `sret` 进入用户态, 即可在切换后的任务获得一个正确的上下文。

2. L43-L48: 特殊处理了 `sstatus`、`sepc`、`sscratch`。

- **`sstatus`:** 表示 S 态下的一系列状态, 对于进入用户态, 主要是 `SPP` 位记录了 trap 前的特权级。设为 `User` 后, `sret` 执行时 CPU 据此回到 U 态。此外还有中断使能位等 (如 `SPIE/SIE`), 影响返回到用户态后中断状态。
- **`sepc`:** 保存了 trap 要返回的地址 (也就是要回到用户态继续执行的 PC)。
- **`sscratch`:** 暂存用户栈指针。在 L60 的 `csrrw` 中与 `sp` 交换, 使 `sp` 恢复为用户栈。

3. L50-L60: 因为 `x2` 和 `x4` 分别对应 `sp` 和 `tp`。根据这两个寄存器的含义:

- **`x2/sp`:** 此时 `sp` 正被用来索引 `TrapContext`, 不能提前覆盖。用户 `sp` 已经在 L45 被加载到 `sscratch`, 将在 L60 通过 `csrrw` 交换回来。
- **`x4/tp`:** 用户程序不使用 `tp` 寄存器 (注释里写着 `application does not use it`), 且在整个 OS 中 `tp` 未被修改, 无需保存/恢复。

4. L60: 这句 `csrrw` 指令执行了一次 `sp` 与 `sscratch` 的原子交换。执行前: `sp` = 内核栈顶 (`addi sp, sp, 34*8` 释放了 `TrapContext`), `sscratch` = 用户栈指针, 因此执行后:

- **`sp` = 用户栈指针**
- **`sscratch` = 内核栈顶**

5. `__restore`: 发生在 L61: `sret`。这是 `sret` 指令本身的语义带来的结果, 具体来说, `sret` 指令在 CPII 中会干如下事情:

sepc 寄存器中保存的下一条指令地址。

- 将 PC 设为 `sepc` 的值，跳转到用户程序
 - 根据 `sstatus.SPP` 位切换特权级，因为 L46 已将 `sstatus` 恢复为 `SPP=User`，所以 `sret` 执行后 CPU 从 S 态进入 U 态。
6. L13: 这是 `__alltraps` 的第一条指令。执行前：`sp` = 用户栈指针，`sscratch` = 内核栈顶。类比第4小题，这里原子交换后：
- `sp` = 内核栈顶
 - `sscratch` = 用户栈指针
7. 从 U 态进入 S 态: 是在 L38: `call trap_handler` 发生的，这一步用户态程序通过 `ecall` 指令执行系统调用，随后 CPU 会做将当前 PC 保存到 `sepc`、将当前特权级保存到 `sstatus.SPP` 与切换状态等一系列动作。

荣誉准则

1. 在完成本次实验的过程（含此前学习的过程）中，我曾分别与 无 以下各位 就（与本次实验相关的）以下方面做过交流，还在代码中对应的位置以注释形式记录了具体的交流对象及内容：

《你交流的对象说明》：无

2. 此外，我也参考了 无 以下资料，还在代码中对应的位置以注释形式记录了具体的参考来源及内容：

《你参考的资料说明》：无

3. 我独立完成了本次实验除以上方面之外的所有工作，包括代码与文档。我清楚地知道，从以上方面获得的信息在一定程度上降低了实验难度，可能会影响起评分。
4. 我从未使用过他人的代码，不管是原封不动地复制，还是经过了某些等价转换。我未曾也不会向他人（含此后各届同学）复制或公开我的实验代码，我有义务妥善保管好它们。我提交至本实验的评测系统的代码，均无意于破坏或妨碍任何计算机系统的正常运转。